- RAG Learning

3.1 Fundamentals of Algorithms

AQA GCSE Computer Science (8525) — Paper 1

3.1.1 Representing Algorithms

> An **algorithm** is a precise, step-by-step sequence of instructions for solving a problem or completing a task in a finite amount of time. Algorithms are **language-independent** — they describe *what* has to be done, not *how* a particular programming language expresses it. A **program** is the implementation of an algorithm in a specific programming language that a computer can execute.

Think of a cooking recipe. The recipe itself (the list of steps: chop onions, add oil, heat pan, stir for 5 minutes…) is the algorithm. Rewriting the same recipe in French, or reading it aloud, or printing it in a book does not change the algorithm — it only changes the *representation*. A program is like that recipe written out in a language the kitchen staff can follow precisely, such as Python, C#, or AQA pseudo-code.

Every algorithm must have certain properties to be valid. It must be **precise** (each step is unambiguous), **finite** (it eventually terminates rather than running forever), and must produce the same correct output whenever it is given the same input (it is **deterministic**). These properties are what distinguish an algorithm from a vague plan — for example, "make tea" is a plan, but "boil 200 ml of water, add one teabag, wait 2 minutes, remove the teabag, add 20 ml of milk" is an algorithm.

Algorithm vs Program — The Key Distinction

Algorithm	Program
A logical sequence of steps to solve a problem	The algorithm written in a specific programming language

Language-independent — can be shown as flowchart, pseudo-code, English, or structured diagram	Language-dependent — written in Python, Java, C#, etc.
Cannot be run directly by a computer	Can be run (after translation by an interpreter or compiler)
Focuses on the **logic** of the solution	Focuses on the **implementation** (syntax, data types, libraries)
Example: "Keep asking the user for a number until they enter 0, then output the total."	Example: 12 lines of Python that read inputs, accumulate a total, and print it.

> **Exam Tip:** A very common exam question is to "state the difference between an algorithm and a program." The mark-winning phrase is: an algorithm is a *language-independent set of steps* to solve a problem, whereas a program is the *implementation of an algorithm in a specific programming language*. The same algorithm can be implemented as many different programs.

Flowcharts

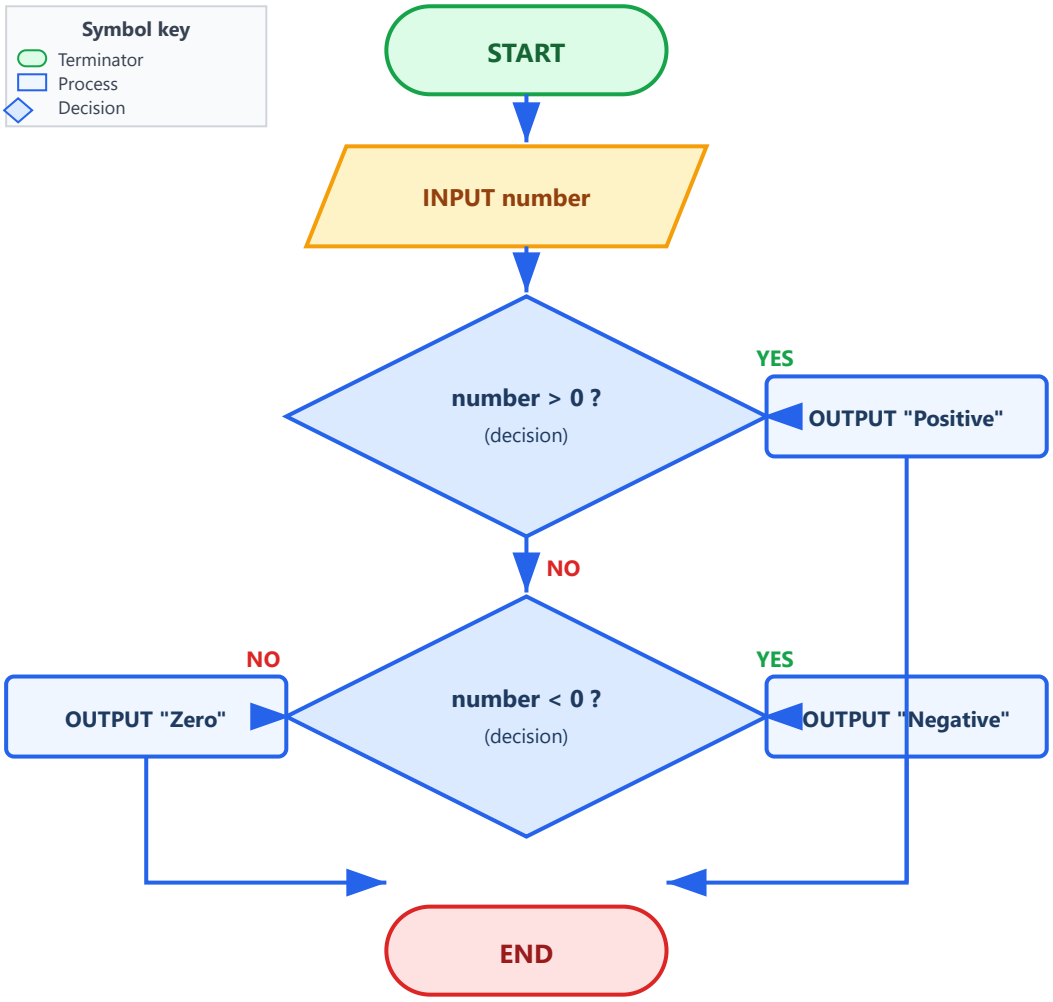
A **flowchart** is a visual way of representing an algorithm using a set of standardised symbols connected by arrows. The arrows show the order (flow) in which the steps are carried out. Flowcharts are useful because they make the structure of a program easy to see at a glance — decisions, loops, and parallel paths are obvious.

AQA requires that you know and can use the following four standard flowchart symbols:

Symbol	Shape	Meaning	Example
Terminator	Rounded rectangle (stadium / oval)	Marks the **start** or **end** of an algorithm. Every flowchart has exactly one Start and at least one End.	`START`, `END`, `STOP`
Process	Rectangle	A single step of **processing** — an assignment, a calculation, or any action that transforms data.	`total ← total + price`

Decision	Diamond (rhombus)	A **question** with a Yes/No (or True/False) answer. Two arrows leave the diamond, one for each outcome.	`is x > 10?`
Input/Output	Parallelogram	Data being **read in** (from the user/a file/a sensor) or **output** (to the screen/a printer/a file).	`INPUT age`, `OUTPUT total`

Flowchart: Is a Number Positive, Negative or Zero?



Pseudo-code

Pseudo-code is a structured, language-like way of writing an algorithm in plain English. It looks a bit like a programming language (with keywords such as IF, WHILE, FOR) but does not follow the strict syntax rules of any real language. Pseudo-code is useful because it expresses the *logic* of an algorithm without the student having to worry about missing semicolons or indentation errors.

AQA provides a **standard pseudo-code reference** for its exams. Candidates do not have to use exactly AQA's style, but the style used must be *clear and consistent*. Below are the main conventions used in AQA pseudo-code and in this guide:

- **Assignment**: `x ← 5` or `x = 5` (a value is assigned to a variable)
- **Input / output**: `INPUT name` or `USERINPUT`, and `OUTPUT total`
- **Selection**: `IF condition THEN … ELSE … ENDIF`
- **Iteration**: `WHILE … ENDWHILE`, `REPEAT … UNTIL`, `FOR i ← 1 TO n … ENDFOR`
- **Comments**: `# this is a comment`
- **Array access**: `list[0]`, `list[i]` (AQA uses zero-based indexing)

```
# Pseudo-code for the same algorithm as the flowchart above
INPUT number
IF number > 0 THEN
    OUTPUT "Positive"
ELSE
    IF number < 0 THEN
        OUTPUT "Negative"
    ELSE
        OUTPUT "Zero"
    ENDIF
ENDIF
```

Hand-Tracing with Trace Tables

> **Hand-tracing** (or "dry running") an algorithm means stepping through it line by line with specific example inputs, recording how each variable changes at every step. A **trace table** is a table with one column per variable and one row per step, used to record those changes. Trace tables are used to find logic errors, to predict the output of an algorithm, and to verify that an algorithm works as intended.

A trace table has:

- One **column for each variable** whose value changes (plus often a column for any OUTPUT).
- One **row for each change** (each time a variable is updated).

- An optional **line number** column showing which line of the algorithm caused the change.

To complete a trace table, you start at the top of the algorithm and write down each variable's starting value. As you work through each line, you update the values, adding a new row only when a value actually changes. Unchanged columns are left blank on subsequent rows (you do not repeat a value that did not change). At every INPUT, record the given input; at every OUTPUT, record the value printed.

Worked Example — Trace Table

```
# Algorithm: sum the numbers 1 + 2 + 3 + 4 + 5
total ← 0
FOR i ← 1 TO 5
    total ← total + i
ENDFOR
OUTPUT total
```

Line	i	total	OUTPUT
1 (initialise)	—	0	
2 (FOR start)	1		
3 (total + i)		1	
2 (next i)	2		
3 (total + i)		3	
2 (next i)	3		
3 (total + i)		6	
2 (next i)	4		
3 (total + i)		10	
2 (next i)	5		

© RAG Learning · Not for redistribution

3 (total + i)		15	
5 (OUTPUT)			15

Exam Tip: In a trace-table question, always include a column for every variable *and* a column for OUTPUT if the algorithm produces one. Only add a new row when a value actually changes — do not repeat unchanged values. If there is an INPUT instruction, the question will tell you what the user entered — write it in the matching variable column. A neat, complete trace table is often worth 4+ marks on its own.

3.1.2 Efficiency of Algorithms

The **efficiency** of an algorithm is a measure of how much resource it uses to solve a problem — usually either **time** (how long it takes) or **space** (how much memory it uses). Two algorithms can solve exactly the same problem but differ enormously in efficiency: one might take a fraction of a second, the other might take hours on the same data.

There is almost always more than one way to solve a problem. For example, to find a name in a contact list you could: scan the list from top to bottom; jump to a known index; use the first letter as a hint; or, if the list is sorted, use a clever halving strategy. Every one of these is a valid algorithm, but they are not equally *efficient*. Comparing algorithms by efficiency is how computer scientists choose the best one for a given situation.

At GCSE level, we compare time efficiency *informally* — usually by counting the **number of basic steps** (or **comparisons**) an algorithm performs on inputs of different sizes. We do not use formal Big-O notation in AQA 8525, but you must be able to reason in the same spirit: if doubling the input size doubles the work, that is a linear relationship; if doubling the input size adds only one extra step, that is much better; if doubling the input size quadruples the work, that is much worse.

Counting Steps — A Worked Comparison

Suppose we want to find the largest number in a list of **n** items. Here are two algorithms that both work:

```
# Algorithm A: compare every pair of items
largest ← list[0]
FOR i ← 0 TO n-1
    FOR j ← 0 TO n-1
        IF list[j] > largest THEN
            largest ← list[j]
        ENDIF
    ENDFOR
ENDFOR
OUTPUT largest
```

```
# Algorithm B: one pass through the list
largest ← list[0]
FOR i ← 1 TO n-1
    IF list[i] > largest THEN
        largest ← list[i]
    ENDIF
ENDFOR
OUTPUT largest
```

List length (n)	Comparisons in A (n×n)	Comparisons in B (n−1)
10	100	9
100	10,000	99
1,000	1,000,000	999
1,000,000	1,000,000,000,000	999,999

Algorithm A does the same comparisons over and over again in an unnecessary nested loop. Algorithm B goes through the list only once. Both produce the correct answer, but B is **enormously more time-efficient**. For a list of a million items, Algorithm A might take minutes; Algorithm B finishes in a fraction of a second.

> **Exam Tip:** When a question asks you to "compare the efficiency" of two algorithms, don't just say "one is faster". Give a reason tied to the number of steps or comparisons on an input of

size n. For example: "Algorithm B performs n comparisons whereas Algorithm A performs roughly n^2. On a list of 1000 items A does 1,000,000 comparisons but B only does 999, so B is much faster for large lists."

3.1.3 Searching Algorithms

A **search algorithm** finds whether a given item exists in a collection of data, and usually where it is. AQA requires you to know two searches: **linear search** and **binary search**.

Linear Search

Linear search checks each item in the list **one at a time, from start to end**, comparing each with the target value. It stops as soon as it finds a match (or when it has checked every item without finding one). Linear search works on **any** list — the data does *not* have to be sorted.

Imagine searching for a friend's phone number in an un-sorted pile of business cards: you have no choice but to flip through the cards one by one. That is linear search.

```
# Linear search pseudo-code
FUNCTION linearSearch(list, target)
    FOR i ← 0 TO length(list) - 1
        IF list[i] = target THEN
            RETURN i          # found — return the index
        ENDIF
    ENDFOR
    RETURN -1                 # not found
ENDFUNCTION
```

When to Use Linear Search

- The list is **unsorted** (binary search requires sorted data).
- The list is **small** — with only a handful of items, the speed difference is negligible.
- You only need to run the search **once or twice** — sorting first would take longer than simply scanning.

- The data is stored in a structure that does not allow random access (e.g. a linked list, a stream of sensor data), where jumping to the middle element is not possible.

Efficiency of Linear Search

For a list of **n** items, linear search performs:

- **Best case:** 1 comparison (the item is at position 0).
- **Worst case:** *n* comparisons (item is last, or not present).
- **Average case:** about *n*/2 comparisons.

The number of comparisons grows **linearly** with the size of the list — doubling the list doubles the worst-case work. In informal terms, linear search has **O(n)** time complexity, but you are not required to write it that way at GCSE.

Binary Search

> **Binary search** repeatedly **divides the search space in half**. It looks at the middle item of the list; if that is the target, it has found it; if the target is smaller, it discards the right half and searches only the left half; if the target is larger, it discards the left half and searches only the right half. This halving continues until the target is found or the search space is empty. **Binary search only works on a list that is already sorted.**

Imagine looking up a word in a paper dictionary. You do not start at page 1 and read every word — you open the dictionary roughly in the middle, see whether your word would come before or after, and then repeat the same process on just that half. Binary search is the computer-science version of this highly efficient strategy.

```
# Binary search pseudo-code (iterative)
FUNCTION binarySearch(list, target)
    low ← 0
    high ← length(list) - 1
    WHILE low ≤ high
        mid ← (low + high) DIV 2
        IF list[mid] = target THEN
            RETURN mid
        ELSE IF list[mid] < target THEN
            low ← mid + 1        # target must be in the right half
```

```
        ELSE
            high ← mid - 1       # target must be in the left half
        ENDIF
    ENDWHILE
    RETURN -1                        # not found
ENDFUNCTION
```

Efficiency of Binary Search

Each pass halves the remaining search space. If the list has *n* items, the maximum number of comparisons needed is roughly **$\log_2(n)$** (the number of times you can halve *n* before reaching 1):

List size (n)	Max comparisons (linear search)	Max comparisons (binary search)
8	8	3
100	100	7
1,000	1,000	10
1,000,000	1,000,000	20
1,000,000,000	1,000,000,000	30

The difference is dramatic for large lists: even a billion-item list needs only around 30 comparisons! Informally, binary search has **O(log n)** time complexity — growth is logarithmic rather than linear.

Linear vs Binary Search — Comparison

Feature	Linear Search	Binary Search
Data must be sorted?	No — works on any list	**Yes** — only works on a sorted list
How it works	Checks every item in order from start to end	Repeatedly halves the search space by comparing to the middle item
Best case	1 comparison (first item)	1 comparison (middle item is the target)
Worst case	*n* comparisons	$\log_2(n)$ comparisons (about 20 for a million items)
Speed on large lists	Slow — grows linearly	Very fast — grows logarithmically
Simplicity	Very simple to write	A little more complex (need low/high/mid pointers)
Best used when	List is small or unsorted, or search is done once	List is large and already sorted, or many searches are needed

Exam Tip: Two absolute must-remember facts about binary search: (1) the list **must be sorted** first — this is worth a mark on almost every binary-search question; (2) it is efficient because each step **halves** the remaining search space. Examiners love candidates who explicitly say "halves" rather than vaguely "reduces" or "narrows down".

3.1.4 Sorting Algorithms

A **sorting algorithm** puts the items in a list into order (usually ascending). Sorted data is easier to search (binary search requires it), easier to present to users, and often easier to process. AQA requires you to know **bubble sort** and **merge sort**.

Bubble Sort

> **Bubble sort** works by repeatedly going through the list, **comparing adjacent pairs** of items and **swapping them** if they are in the wrong order. Each complete left-to-right scan is called a **pass**. After each pass, the largest remaining unsorted item has "bubbled" to the end of the list and is in its correct final position. The algorithm finishes when a complete pass is made with *no swaps*, meaning the list is sorted.

```
# Bubble sort pseudo-code
FUNCTION bubbleSort(list)
    n ← length(list)
    REPEAT
        swapped ← False
        FOR i ← 0 TO n - 2
            IF list[i] > list[i + 1] THEN
                # swap
                temp ← list[i]
                list[i] ← list[i + 1]
                list[i + 1] ← temp
                swapped ← True
            ENDIF
        ENDFOR
        n ← n - 1                  # optimisation: last item is now in place
    UNTIL swapped = False
    RETURN list
ENDFUNCTION
```

Bubble Sort: One Complete Pass over [5, 2, 8, 1, 4]

Each step compares an adjacent pair and swaps if the left is greater. After one pass, the largest value (8) has bubbled to the end.

Start	5	2	8	1	4	Compare 5 and 2. 5 > 2 so swap.
Step 1	2	5	8	1	4	Compare 5 and 8. 5 < 8, no swap.
Step 2	2	5	8	1	4	Compare 8 and 1. 8 > 1 so swap.
Step 3	2	5	1	8	4	Compare 8 and 4. 8 > 4 so swap.
End of Pas	2	5	1	4	8	The largest value (8) is now at the end.

Pass 1 complete:
4 comparisons, 3 swaps. Largest (8) now in its final position.
The algorithm repeats with the remaining unsorted 4 items. After 4 passes the list is fully sorte

Efficiency of Bubble Sort

For a list of **n** items:

- Pass 1 makes up to **n−1** comparisons.
- Pass 2 makes up to **n−2** comparisons (the last item is now sorted).
- … and so on, down to 1 comparison in the final pass.
- Total maximum comparisons = (n−1) + (n−2) + … + 1 = **n(n−1)/2**, roughly $\mathbf{n^2/2}$.

Doubling the list size quadruples the work. Informally this is $\mathbf{O(n^2)}$. That makes bubble sort **slow on large lists**. On a list of 10 items it does at most 45 comparisons — fine. On a list of 10,000 items it does around 50,000,000 comparisons — very slow.

Merge Sort

> **Merge sort** is a **divide-and-conquer** sorting algorithm. It repeatedly divides the list in half until every sub-list contains just one item (a list of one is by definition sorted). It then **merges** the sub-lists back together in the correct order, two at a time, producing a single fully sorted list. Merge sort is naturally **recursive** — each half is sorted by calling merge sort on it.

Merge sort has two phases:

1. **Divide:** split the list in half, then split each half in half, and keep going until every piece is a list of size 1.
2. **Conquer (merge):** take two already-sorted adjacent pieces and combine them into one larger sorted piece. Keep merging pairs until only one list remains.

How Merging Works

Given two sorted sub-lists (say `[2, 5, 8]` and `[1, 4, 7]`), compare the first item of each. Take the smaller one and put it in the output, and move the pointer on that sub-list forward. Repeat until one sub-list is empty, then copy across whatever is left of the other sub-list.

```
# Merging two sorted sub-lists into a single sorted list
left  = [2, 5, 8]
right = [1, 4, 7]

compare 2 and 1 → take 1 → [1]
compare 2 and 4 → take 2 → [1, 2]
compare 5 and 4 → take 4 → [1, 2, 4]
compare 5 and 7 → take 5 → [1, 2, 4, 5]
compare 8 and 7 → take 7 → [1, 2, 4, 5, 7]
right is empty → copy rest of left → [1, 2, 4, 5, 7, 8]
```

```
# Merge sort pseudo-code
FUNCTION mergeSort(list)
    IF length(list) ≤ 1 THEN
        RETURN list                              # already sorted
    ENDIF
    mid   ← length(list) DIV 2
    left  ← mergeSort(list[0 : mid])          # recursive call on left half
    right ← mergeSort(list[mid : length(list)]) # recursive call on right half
    RETURN merge(left, right)                    # combine the two sorted halves
ENDFUNCTION

FUNCTION merge(left, right)
    result ← []
    i ← 0
    j ← 0
    WHILE i < length(left) AND j < length(right)
        IF left[i] ≤ right[j] THEN
            result.append(left[i])
            i ← i + 1
```

```
        ELSE
            result.append(right[j])
            j ← j + 1
        ENDIF
    ENDWHILE
    # append whatever is left
    WHILE i < length(left)
        result.append(left[i])
        i ← i + 1
    ENDWHILE
    WHILE j < length(right)
        result.append(right[j])
        j ← j + 1
    ENDWHILE
    RETURN result
ENDFUNCTION
```

Worked Example: Merge Sort on [6, 3, 8, 1, 5, 2]

```
DIVIDE phase (keep splitting until size = 1):
    [6, 3, 8, 1, 5, 2]
      /             \
   [6, 3, 8]       [1, 5, 2]
    /    \          /    \
  [6]   [3, 8]    [1]   [5, 2]
         /  \            /  \
       [3]  [8]         [5]  [2]

MERGE phase (combine pairs in order):
  [3] + [8]  → [3, 8]
  [5] + [2]  → [2, 5]
  [6] + [3, 8]  → [3, 6, 8]
  [1] + [2, 5]  → [1, 2, 5]
  [3, 6, 8] + [1, 2, 5]  → [1, 2, 3, 5, 6, 8]   (SORTED)
```

Efficiency of Merge Sort

Merge sort has **$\log_2(n)$** levels of division (because the list is halved each time), and each level of merging takes roughly **n** steps (every item has to be placed into its merged list). The total work is therefore proportional to **$n \times \log_2(n)$** — informally **O(n log n)**. That is enormously faster than bubble sort's n^2 on large lists:

List size (n)	Bubble sort (≈ $n^2/2$)	Merge sort (≈ $n \log_2 n$)
10	45	33
100	4,950	~664
1,000	499,500	~9,966
1,000,000	~500,000,000,000	~19,931,569

Bubble vs Merge Sort — Comparison

Feature	Bubble Sort	Merge Sort
How it works	Repeatedly compares and swaps adjacent pairs	Divides list in half recursively, then merges sorted sub-lists
Design style	Iterative (loops)	Recursive (divide and conquer)
Simplicity to implement	Very simple (a nested loop and a swap)	More complex — needs recursion and a separate merge function
Speed on small lists	Fine — overhead is low	Also fine, but recursion adds slight overhead
Speed on large lists	Very slow (n^2)	Much faster (n log n)
Memory usage	Very low — sorts "in place"	Higher — temporary lists are created during the merge step
Best case	Fast if list is already sorted (a single pass with no swaps)	Always n log n — no short-cut for already-sorted lists
Best used when	Lists are small, or the data is nearly sorted	Lists are large and time efficiency is important

© RAG Learning · Not for redistribution

Exam Tip: If an exam asks "which sort is more efficient for a large list" the answer is almost always merge sort (n log n versus bubble sort's n^2). But if it asks "which is simpler to implement" or "which uses less memory", the answer is bubble sort. Read the question carefully — both sorts have situations where they are the right choice.

Exam Tip: Remember: merge sort is recursive — it calls itself on each half of the list. The base case (stopping condition) is a list of 0 or 1 items, which is already sorted. Missing or wrong base cases are a common programming mistake with recursion.

3.1.5 Decomposition and Abstraction

Real-world problems are often far too complex to solve in one go. Professional programmers use two key mental tools to tackle big problems: **decomposition** and **abstraction**. Both are fundamental computational-thinking skills and are tested on every AQA paper.

Decomposition

Decomposition is breaking a large, complex problem down into smaller, more manageable sub-problems. Each sub-problem can then be solved individually, tested independently, and finally combined to solve the original problem. Decomposition makes a big problem tractable and allows a team of people to work on different sub-problems in parallel.

Suppose you are asked to design a computer game. "Write a game" is far too big a task to start on all at once — there are a thousand things to consider. But if you decompose it, you can split the problem into sub-problems such as: player input, game logic, graphics/rendering, sound, high-score saving, menu screens. Each of these is a smaller, well-defined problem that can be solved on its own.

Example of Decomposition — Designing an Online Shop

- **Sub-problem 1:** Display product catalogue (with categories, search, images).
- **Sub-problem 2:** Manage user accounts (register, log in, view order history).
- **Sub-problem 3:** Shopping cart (add/remove items, update quantities, total calculation).

- **Sub-problem 4:** Payment processing (verify card, communicate with payment gateway).
- **Sub-problem 5:** Order fulfilment (generate packing slip, email confirmation, update stock).

Each of those sub-problems can itself be decomposed further — for example "Shopping cart" can be broken into "add item function", "remove item function", "update quantity function", "calculate total function", and so on. This hierarchical breakdown continues until every sub-problem is simple enough to be coded as a single function.

Abstraction

> **Abstraction** is the process of removing or hiding unnecessary details from a problem so you can focus on the key features that matter for the solution. An abstraction is a simpler model of something complex that keeps only the information you need for the task at hand.

Think of the London Underground map. The actual geography of London is enormously complicated — stations are at specific latitudes and longitudes, tracks curve between them, and the true distances vary. But when you are travelling, you only need to know: which line goes to which station, and where the interchanges are. The famous Tube map *abstracts away* real distances, real angles and real geography, keeping only the information a passenger needs. It is useless as a street map, but perfect as a travel map.

Good abstractions:

- Keep only the **relevant details** for the task being solved.
- Hide details that would be **distracting or overwhelming**.
- Allow the problem to be **understood more easily** and solved more cleanly.

Example of Abstraction — Representing a Student
For a school register program, a "Student" might be represented with only the fields `name`, `form`, `present`. Their favourite food, hair colour, height, favourite band — all irrelevant to taking a register. They are abstracted away. The same student in a medical records system would have a totally different abstraction: `NHS_number`, `allergies`, `date_of_birth`, `medical_history`. Name is still relevant, but "favourite band" is not. Abstraction is always relative to *what you are trying to do*.

Example of Abstraction — Driving a Car

You learn to drive by learning the steering wheel, the pedals, and the gear stick. You don't need to know how the fuel injectors meter fuel, how the differential works, or how the engine control unit adjusts ignition timing. The controls are an *abstraction* of the mechanical reality of the car, simple enough for anyone to use.

Decomposition vs Abstraction — The Difference

Decomposition	Abstraction
Breaks a problem into smaller sub-problems	Removes unnecessary detail so you can focus on what matters
Answers: "What are the parts of this problem?"	Answers: "What details can I ignore?"
Produces a list/tree of sub-tasks	Produces a simpler model of the problem
Makes a big task more manageable	Makes a complex task simpler to reason about
Example: an online shop split into cart, login, payment, stock modules	Example: a Tube map showing station connections but not real distances

Putting Them Together — Designing a Sat-Nav

Imagine designing a satellite-navigation program.

- **Decomposition** splits the task into sub-problems: GPS signal reading, map rendering, route calculation, turn-by-turn voice instructions, user-interface.
- **Abstraction** removes unnecessary detail: the map does not need to show every building or tree. The route calculator treats the road network as a simple graph of junctions connected by roads with a "cost" (time or distance) — the actual physical road surface, traffic lights, and so on are abstracted away.

Decomposition and abstraction work together: you decompose to split the problem up, and abstract within each part to keep it simple. Together they are the foundation of almost all modern software design.

> **Exam Tip:** Students often confuse decomposition and abstraction. The quickest mnemonic: decomposition splits a problem *horizontally* (into smaller sibling sub-problems); abstraction

simplifies a problem *vertically* (by removing detail). A question that says "break down a problem" is about decomposition. A question that says "remove unnecessary detail" or "simplify" is about abstraction. Giving a clear example is usually worth a mark on its own.

Exam Tip: When asked to apply decomposition to a scenario (e.g. a school timetable system), list at least three concrete sub-problems and briefly say what each does. When asked to apply abstraction, explicitly name *which details are kept* and *which are ignored*, and why that is sensible for the task.

© RAG Learning · Not for redistribution